

Baseline11 Auto Correlation Tester Guide

From Postman collection execution to a validated JMeter 5.6.3 test plan

Purpose. This guide explains how a tester uses Baseline11 Auto Correlate to execute a working Postman collection twice with Newman, upload the resulting JSON reports, confirm dynamic producer and consumer relationships, generate a correlated JMX, and validate it with Apache JMeter.

Expected outcome. The tester receives a structurally valid JMX in which dynamic response values are extracted and reused automatically, while credentials remain external JMeter properties unless the tester explicitly chooses to embed them.

Document	Audience	Version	Date
Operating guide	API, QA and performance testers	1.0	23 September 2026

What the tester needs

- A Postman collection that already executes successfully.
- A Postman environment or another safe source for host and credential values.
- Newman and the optional htmlextra reporter.
- Two successful Newman JSON reports from the same collection and request order.
- Access to the Baseline11 Auto Correlate web application and Apache JMeter 5.6.3.

Quick Start

Use this page when the collection, environment and credentials are already prepared. The detailed procedure begins on the next page.

Step	Action	Expected result
1	Verify the collection	Run the selected folder in Postman or Newman and confirm that producer requests succeed.
2	Create baseline	Run Newman and export baseline.json.
3	Create comparison	Run the same command again and export comparison.json.
4	Upload	Upload baseline first and comparison second.
5	Review	Confirm healthy runs, aligned requests and the expected candidate.
6	Correlate	Click Auto correlate all reused values once.
7	Generate	Preview, generate and download the JMX and manifest.
8	Validate	Supply runtime secrets and execute the plan with JMeter 5.6.3.

The Core Concept

Newman does not convert a Postman collection into JSON. Newman executes the collection and records the resolved requests, responses, status codes, assertions and timings in a JSON report. Baseline11 compares two reports to prove which response values are dynamic and where later requests reuse them.

```
Postman collection + environment
|
+--> Newman baseline run --> baseline.json
+--> Newman comparison run --> comparison.json
|
+--> Baseline11 --> correlated JMX --> JMeter
```

Why Two Runs

A single run can show that a value is reused, but it cannot reliably distinguish a dynamic identifier from static configuration. Two successful runs show that the value changes while the producer response path and consumer request locations remain stable.

1 Prepare the Postman Collection

The collection must complete the intended API journey before it is uploaded indirectly through Newman reports. Baseline11 translates an observed working flow into JMeter; it does not repair authentication failures or unresolved Postman placeholders.

Collection readiness checklist

- Requests are ordered in the same sequence the business flow requires.
- Authentication resolves from the selected environment.
- Producer requests return successful status codes and usable response bodies.
- Postman scripts capture values required for the collection itself to continue.
- Later requests use Postman variables such as `{{order_id}}` instead of literal `{order_id}` text.
- Assertions fail when a required producer response or identifier is missing.

Example dynamic flow

Create Order returns:

```
{
  "id": "order_ABC123",
  "amount": 10000,
  "status": "created"
}
```

A Postman post-response script keeps the collection executable:

```
const response = pm.response.json();
pm.test("Order ID was returned", function () {
  pm.expect(response.id).to.be.a("string").and.not.empty;
});
pm.collectionVariables.set("order_id", response.id);
```

A downstream Postman request uses:

```
GET https://api.example.com/v1/orders/{{order_id}}
```

Environment guidance

Keep real keys and secrets in a local environment file. Share only a template with empty values. Confirm that exported environment values are populated before running Newman; some Postman workflows export variable definitions without the local values.

2 Generate Newman Reports

Install the command line tools

```
npm install -g newman newman-reporter-htmlextra  
newman --version
```

Generic baseline command

```
newman run "C:\QA\collection.postman_collection.json" ^  
-e "C:\QA\test.postman_environment.json" ^  
--folder "Business Flow" ^  
-r cli,json,htmlextra ^  
--reporter-json-export "C:\QA\reports\baseline.json" ^  
--reporter-htmlextra-export "C:\QA\reports\baseline.html"
```

Run the same command again with comparison output names:

```
newman run "C:\QA\collection.postman_collection.json" ^  
-e "C:\QA\test.postman_environment.json" ^  
--folder "Business Flow" ^  
-r cli,json,htmlextra ^  
--reporter-json-export "C:\QA\reports\comparison.json" ^  
--reporter-htmlextra-export "C:\QA\reports\comparison.html"
```

Command reference

Argument	Purpose
newman run	Executes a Postman collection.
-e	Loads a Postman environment.
--folder	Limits execution to one folder or scenario.
-r cli,json,htmlextra	Prints terminal output and creates JSON and HTML reports.
--reporter-json-export	Writes the machine-readable report consumed by Baseline11.
--reporter-htmlextra-export	Writes a human-readable execution report for review.

Report acceptance criteria

- Both runs use the same collection, folder and request order.
- The producer and intended consumers return successful responses.
- There are no authentication, network or missing-response failures.
- Required assertions pass.
- Dynamic values differ between the two runs.

3 Upload and Review the Analysis

Open Baseline11 and upload baseline.json first and comparison.json second. Do not upload the Postman collection, environment, HTML report or a JMX into the report uploader.

Run Health

Run Health verifies transport success, assertion results, business response heuristics and sequence alignment. A warning is reviewable; a blocker means the reports are not safe for automatic correlation.

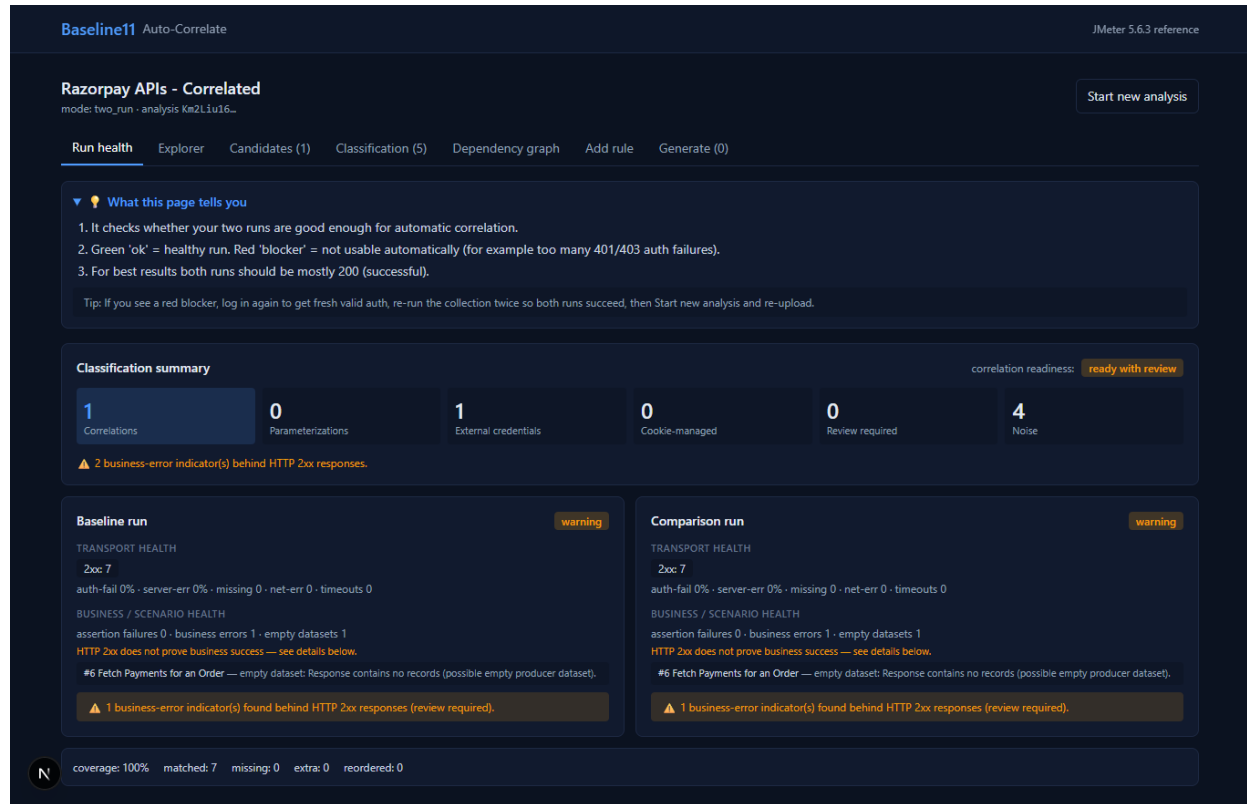


Figure 1 Example Run Health with seven aligned HTTP 200 requests

How to interpret common health results

Result	Meaning	Tester action
Ready	Both reports are suitable for analysis.	Continue to Candidates.
Ready with review	Transport is usable but a business heuristic needs review.	Read the warning and continue only when the response is valid for the scenario.
Blocker	Authentication, response availability or alignment is too poor.	Fix the collection or credentials and create two new reports.
Empty dataset	A successful response returned no records.	Confirm whether an empty result is expected, such as payments for a new unpaid order.

4 Review and Accept Correlation

Candidates are response values that changed between runs and were reused by later requests in both runs. Inspect the producer path, consumer count, confidence and evidence before acceptance.

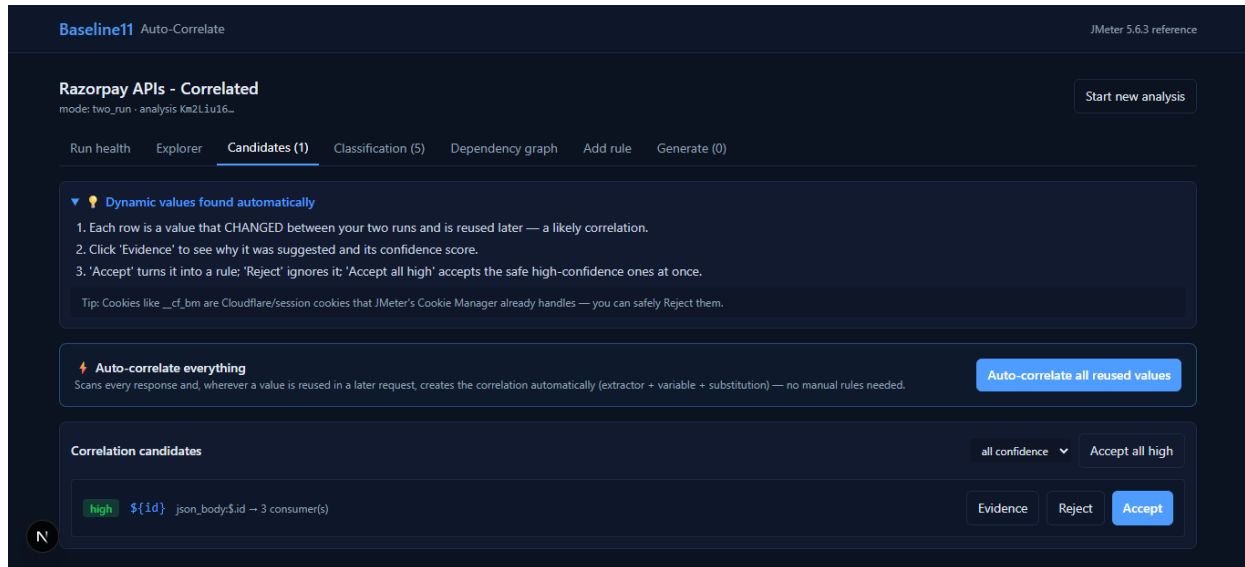


Figure 2 One high-confidence ID candidate produced at JSONPath \$.id and reused by three requests

Decision rules

Observation	Decision
Earlier response produces the value and later requests reuse it	Accept or use automatic correlation.
Value changes but no earlier response produces it	Treat it as input parameterization or runtime configuration.
Authorization, API key or secret	Keep it as an external JMeter property.
Cookie maintained by session behavior	Use JMeter HTTP Cookie Manager.
Host, Content-Length, Postman-Token or Accept-Encoding	Treat as transport noise and let JMeter compute it.

Automatic option

1. Click Auto correlate all reused values once.
2. Confirm the button changes to Correlation completed and becomes disabled.
3. Open Dependency graph and verify the producer points to the expected consumers.
4. Confirm Generate displays the accepted rule count.

Variable names may be edited for clarity. For example, a response field named id can be renamed from \${id} to \${order_id} when the business meaning is an order identifier.

5 Generate the JMeter Plan

Open Generate after accepting the correlation. Use Preview Draft to inspect the planned producer, extractor, variable and consumers before creating the JMX.

Recommended generation settings

Setting	Recommended value	Reason
Threads	1 for functional validation	Reduces diagnostic noise before load sizing.
Loops	1 for functional validation	Confirms one complete business journey.
Parameterize host	Enabled	Keeps the target environment configurable.
Cache Manager	Enabled when browser-like caching is desired	Replays common HTTP caching behavior.
Keep User-Agent	Disabled unless required	Lets JMeter manage transport headers.
Embed static secrets	Disabled	Prevents credentials from being stored in the JMX.

The screenshot shows the 'Preview & generate (1 rules)' interface. At the top, there are settings for threads (1), loops (1), and checkboxes for 'parameterize host' (checked), 'cache manager' (checked), 'keep User-Agent' (unchecked), and 'embed static secrets (opt-in)' (unchecked). Below these are 'Preview Draft' and 'Generate JMX' buttons. A status message indicates 'Generated JMX — structurally valid, not yet executed by JMeter' and 'Structural check: passed'. A summary table shows: 7 matched requests, 1 automatic candidates found, 1 automatic rules accepted, 0 manual rules added, 0 rules edited, 0 rules deleted, 1 confirmed correlations, and 3 downstream values replaced. It also states '9 estimated manual effort saved minutes' and '≈3 minutes of manual JMeter work per replaced occurrence.' At the bottom, there are 'Download Generated JMX' and 'Download manifest' buttons, followed by a section for 'Validate with Apache JMeter 5.6.3' which includes instructions on how to run the plan and a 'Validate with JMeter' button.

Figure 3 Generated JMX summary and runtime Authorization property required for validation

What the JMX should contain

- One HTTP sampler per aligned Newman execution.
- A JSON Extractor attached to the producer request, for example \$.id.
- JMeter variable references such as \${id} or \${order_id} in downstream requests.
- Cookie and cache managers where selected.
- External property expressions such as \${__P(Authorization,,)} for credentials.

6 Validate with Apache JMeter 5.6.3

Generated means the XML structure is valid. Validated means Apache JMeter executed the plan successfully with the required runtime properties. The web validation form passes secrets only to that execution and does not store them in the JMX.

Authorization value for Basic authentication

Supply the complete Basic Authorization header value, not only the key or secret. It has this form:

```
Basic <Base64(api_key:api_secret)>
```

Create the value locally without printing the secret:

```
$e = Get-Content -Raw -LiteralPath 'C:\QA\test.postman_environment.json' | ConvertFrom-Json
$key = ($e.values | Where-Object key -eq 'api_key').value
$secret = ($e.values | Where-Object key -eq 'api_secret').value
$auth = 'Basic ' + [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes($key + ':' + $secret))
```

Open the JMX in the JMeter GUI with runtime credentials

```
& 'C:\Tools\apache-jmeter-5.6.3\bin\jmeter.bat' `
"-JAuthorization=$auth" `
-t 'C:\QA\output\correlated.jmx'
```

For a non-GUI validation run:

```
& 'C:\Tools\apache-jmeter-5.6.3\bin\jmeter.bat' `
-n -t 'C:\QA\output\correlated.jmx' `
"-JAuthorization=$auth" `
-l 'C:\QA\output\results.jtl'
```

Validation acceptance criteria

- All required samplers execute.
- Authentication succeeds and there are no HTTP 401 or 403 responses.
- The producer response is successful.
- The extractor captures a value instead of __NOT_FOUND__.
- Downstream requests contain the extracted runtime value.
- The sampler error ratio meets the project threshold.

7 Package the Deliverables

Give testers the files needed to reproduce the workflow while separating reusable assets from secrets.

File	Share	Purpose
collection.postman_collection.json	Yes	Defines the API workflow.
environment.template.json	Yes	Documents required variables with empty values.
environment.json with real credentials	No	Contains secrets.
baseline.json and comparison.json	Restricted	Contain executed requests, responses and possibly credentials or personal data.
correlated.jmx	Yes when secrets are external	Runs the correlated JMeter workflow.
manifest.json	Yes	Explains variables, extractors, consumers and required properties.
run-jmeter.ps1 template	Yes	Provides a repeatable runtime command without embedded secret values.

Suggested tester package

```
tester-package/  
collection.postman_collection.json  
environment.template.json  
correlated.jmx  
manifest.json  
run-jmeter.ps1  
Baseline11_Auto_Correlation_Tester_Guide.docx
```

Security rules

- Never commit filled environment files, Authorization values or API secrets to source control.
- Treat Newman reports as sensitive until request headers and bodies have been reviewed.
- Keep Embed static secrets disabled for shared or production test plans.
- Use test-mode credentials for demonstrations.
- Rotate a key if it has been exposed in a screenshot, report, ticket or repository.

8 Troubleshooting

Symptom	Likely cause	Corrective action
401 Unauthorized in Newman	Environment values are empty, incorrect, or overridden by request-level auth.	Verify exact variable names and request-level Basic Auth settings, then export the environment again.
URL contains null	The producer failed or its Postman script did not set the variable.	Fix the producer response and assertion before creating reports.
Literal {order_id} remains	The collection uses API-documentation braces instead of Postman syntax.	Use {{order_id}} in the Postman collection.
No candidates	Values did not change, were not reused, or runs are misaligned.	Use two fresh successful runs and verify the downstream request contains the produced value.
Duplicate ID candidates	A later response echoes a value already produced earlier.	Use a current Baseline11 build; the earliest original producer should be retained.
Ready with review	A business heuristic detected an empty or error-like payload behind HTTP 2xx.	Review the response. Continue when the empty result is valid for the scenario.
401 in downloaded JMX	The external Authorization JMeter property was not supplied.	Launch JMeter with -JAuthorization or provide the value in the validation form.
Extractor returns __NOT_FOUND__	The response path or response content differs at runtime.	Inspect the producer response and verify the extractor expression and match number.
Generate shows 0	No candidate has been accepted.	Run automatic correlation or accept the reviewed candidate.

Tester completion checklist

- ☐ Collection completes successfully in Newman.
- ☐ Baseline and comparison JSON reports were generated from the same scenario.
- ☐ Run Health has no blocker.
- ☐ Candidate producer and consumers match the intended business flow.
- ☐ Automatic correlation completed once.
- ☐ Dependency graph shows the expected edges.
- ☐ Generated JMX passes structural validation.
- ☐ JMeter execution succeeds with runtime properties.
- ☐ Extracted variables have real values and are not __NOT_FOUND__.
- ☐ No real credentials are included in the shared package.

Glossary

Term	Meaning
Producer	An earlier response that returns a dynamic value.
Consumer	A later request that reuses the produced value.
Correlation	Capturing a dynamic response value and substituting it into later requests.
Extractor	A JMeter post-processor that reads a value from a response.
Parameterization	Supplying user or environment input that has no response producer.
JMeter property	A runtime value accessed with <code>\$_P(name,)</code> and normally passed with <code>-Jname=value</code> .
Generated JMX	A structurally valid plan that has not necessarily executed successfully.
Validated JMX	A generated plan that Apache JMeter executed within the configured success threshold.

End of tester guide